← Retour index Python Niveau III

Python Niveau III — Syllabus

NumPy • pandas • Contrôle qualité des données

Durée recommandée : **3 jours (21h)** — niveau data intermédiaire avancé.

Objectif général

Passer d'analyses ponctuelles à des pipelines de données reproductibles, contrôlés et exploitables en environnement métier.

Progression pédagogique (du simple au complexe)

Étape 1 — Fondations NumPy

Comprendre arrays, types, axes et vectorisation.

Étape 2 — Fondations pandas

Importer, typer et nettoyer les données tabulaires.

Étape 3 — Transformations métier

Regrouper, joindre, pivoter, calculer des KPI fiables.

Étape 4 — Temporalité et robustesse

Resampling, fenêtres, contrôles de qualité.

Étape 5 — Restitution

Exporter des livrables clairs et documenter les hypothèses.

Définitions essentielles

- **Vectorisation** : calcul appliqué à un tableau complet sans boucle Python explicite.
- **DataFrame** : table de données pandas indexée en lignes/colonnes.
- **Jointure** : combinaison de tables via des clés communes.
- **Cardinalité** : relation entre clés (one-to-one, one-to-many, etc.).
- **KPI** : indicateur métier défini, calculé et suivi dans le temps.

Notions théoriques supplémentaires

- **Copy vs view** : impact mémoire et risques d'écriture involontaire sur les données sources.
- **Indexation avancée** : différences entre `loc`, `iloc`, index hiérarchiques et slicing temporel.
- **Dtypes et coercition** : fiabilité numérique, colonnes catégorielles, précision et NA.
- **Fenêtres glissantes** : moyenne mobile, lissage, signaux de tendance.
- **Idempotence data** : exécuter plusieurs fois un pipeline sans dérive des résultats.
- **Data contracts** : schéma attendu, règles métier et contrôles automatiques.

Extraits de code de référence

NumPy : normalisation

```
import numpy as np
x = np.array([10, 12, 15, 21], dtype=float)
z = (x - x.mean()) / x.std()
```

pandas : nettoyage

```
import pandas as pd

df = pd.read_csv("ventes.csv", parse_dates=["date"])
df["prix_net"] = pd.to_numeric(df["prix_net"], errors="coerce")
df = df.dropna(subset=["prix_net", "client_id"])
df = df[df["prix_net"] >= 0]
```

Consolidation KPI

```
kpi = df.groupby("region", as_index=False).agg(ca=("prix_net", "sum"))
report = kpi.merge(ref_regions, on="region", how="left", validate="one_to_one")
```

Contrôle de schéma minimal

```
expected = {"date": "datetime64[ns]", "region": "object", "prix_net": "float64"}
for col, dtype in expected.items():
    if col not in df.columns:
        raise ValueError(f"colonne manquante: {col}")
    if str(df[col].dtype) != dtype:
        raise ValueError(f"dtype invalide pour {col}: {df[col].dtype}")
```

Compétences visées

- Utiliser NumPy pour accélérer les opérations numériques.
- Nettoyer des jeux de données hétérogènes avec pandas.
- Produire des agrégats fiables avec `groupby`, `merge`, `pivot_table`.
- Mettre en place des contrôles qualité minimaux avant diffusion d'indicateurs.
- Construire un flux d'analyse réutilisable et traçable.

Public cible et prérequis

- Public : analystes, data engineers juniors, développeurs Python orientés data.
- Prérequis : Python intermédiaire + notions de stats descriptives.

Livrables stagiaire

- Notebook/script pipeline complet import → nettoyage → KPI → export.
- Fiche pratique des patterns pandas les plus utiles.
- Checklist de validation de données avant publication.

Glossaire technique approfondi

- **Broadcasting** : mécanisme d'alignement automatique des dimensions NumPy lors d'opérations vectorisées.
- **View** : objet partageant la même mémoire qu'un array source.
- **Copy** : duplication mémoire indépendante de la source.
- **Cardinalité de jointure** : relation attendue entre clés (`1:1`, `1:n`, `n:1`, `n:n`).
- **Resampling** : changement de granularité temporelle (jour, semaine, mois).

- **Outlier** : valeur atypique susceptible de biaiser un agrégat métier.
- **Data drift** : dérive progressive de distribution d'un jeu de données dans le temps.
- **Data lineage** : traçabilité des transformations appliquées de la source au livrable.
- **KPI contract** : définition stable d'un indicateur (formule, périmètre, cadence, exclusions).
- **Idempotence pipeline** : relancer un traitement sans modifier le résultat final attendu.

Détails techniques par domaine

NumPy

- Choisir explicitement les `dtype` pour contrôler précision et empreinte mémoire.
- Privilégier les opérations vectorisées et les ufunc plutôt que les boucles Python.
- Contrôler les copies involontaires lors des slices, reshape et concaténations.
- Valider la forme des tenseurs avant calcul pour éviter les erreurs silencieuses de broadcasting.

pandas

- Imposer un schéma de colonnes et des règles de typage en entrée.
- Tracer les taux de NA, doublons et valeurs hors bornes à chaque étape critique.
- Utiliser `merge(validate=...)` pour détecter les cardinalités non conformes.
- Séparer clairement nettoyage, enrichissement et agrégation dans des fonctions distinctes.

KPI et restitution

- Documenter les exclusions métier (annulations, retours, tests internes).
- Versionner les définitions d'indicateurs pour garantir comparabilité historique.
- Ajouter des contrôles de cohérence inter-indicateurs avant publication.
- Exporter avec métadonnées minimales: date de calcul, période couverte, version de logique.

Pièges techniques fréquents et contremesures

- **Piège** : jointure many-to-many non voulue. **Contremesure** : contraintes de cardinalité explicites + audit des clés.
- **Piège** : conversions de types implicites. **Contremesure** : coercition explicite et rejet des lignes ambiguës.
- **Piège** : KPI « juste » mais non traçable. **Contremesure** : fiche KPI + version de calcul + logs d'exécution.

- **Piège** : notebook non reproductible. **Contremesure** : pipeline scripté avec fonctions déterministes.
- **Piège** : optimisation prématurée sans mesure. **Contremesure** : benchmark avant/après sur cas réel.